

Since you are working with **Angular v21.1.3** in 2026, you are in the era of "Unified Reactivity." Angular has moved away from the complex overhead of zone.js and the boilerplate of RxJS-heavy modules in favor of a **Signal-first** architecture.

Here is the breakdown of the major v21 features and how to refactor your specific application to showcase them.

1. New Features in Angular v21

The biggest shift in v21 is that the framework is now **Zoneless by default** and provides a declarative way to handle asynchronous data and forms.

- **Signal Forms (Experimental):** Replaces ReactiveFormsModule. It uses a `form()` function to create a reactive model directly from signals. No more `FormControl` or `valueChanges` subscriptions.
 - **Zoneless Change Detection:** New applications no longer include zone.js. Change detection is triggered by signal updates, making the app faster and more predictable.
 - **The Resource API (resource & rxResource):** A stable way to handle data fetching. It automatically manages loading states, errors, and request cancellations without manual RxJS logic.
 - **Angular Aria:** A headless accessibility library (Developer Preview) that provides logic for complex UI patterns like grids and listboxes without forcing a specific style.
 - **@let Syntax:** You can now declare local variables directly in templates (e.g., `@let data = mySignal()`).
 - **Native Federation Support:** A standardized way to build Micro Frontends using native browser ESM (ES Modules) and import maps, removing the dependency on Webpack's Module Federation.
 - **Angular MCP Server:** A "Model Context Protocol" integration that allows AI agents to directly understand and help you debug your Angular project structure in real-time.
-

2. Implementation: Modernizing Your Application

To showcase v21, you should transform your app from an **Observable-based monolith** into a **Signal-based Federated Architecture**.

A. Backbone Grid & Data Fetching (Miscellaneous & File Formats)

Your current grid uses RxJS to fetch .json. In v21, you should replace this with the **Resource API**.

- **Modification:** Replace `http.get(...).subscribe()` with `rxResource`.
- **Benefit:** The grid will automatically know if it is `isLoading()`, if there is an `error()`, or if the data is `resolved()` without you writing a single line of state management.

B. Algorithmic State (Dijkstra, Sorting, Hanoi)

These demos rely on frequent state updates.

- **Modification:** Move the coordinate data and disk positions from RxJS Subjects to **Signals**.
- **Hanoi Towers:** Use a `computed()` signal to calculate the valid next move based on the current state signal. This ensures the 2D/3D views stay perfectly synchronized without manual event emitting.
- **Sorting Benchmark:** Use the new **Vitest** (now the default test runner) to benchmark your sorting logic; it is significantly faster than Karma/Jasmine.

C. Forms Migration (OCR, Chat, Regex)

Your forms currently use `ReactiveFormsModule`. This is considered "Legacy" in v21.

- **Modification:** Use the new form function from `@angular/forms/signals`.
- **Example for OCR Form:**

```
TypeScript
// Modern Signal Form
protected readonly ocrForm = form({
  source: signal<'camera' | 'pad'>('camera'),
  result: signal('')
});
```

- **HTML:** Use the `[formField]` directive (which replaced `formControlName`).

D. Scaling with Native Federation

You mentioned splitting the **PDF engine** into a separate program. v21 is the perfect time to implement **Native Federation**.

- **Strategy:** Create a "Remote" application specifically for the C++ / .NET interop (the "Engine").
- **Implementation:** Use `@softarc/native-federation`. The main Angular app (Host) will load the PDF generator or the Neural Network engine on demand. This keeps your main bundle small and allows you to update the "Algorithms" engine without redeploying the UI.

Summary of Refactoring Changes

Feature	Legacy v17 Pattern	Modern v21 Pattern
Data Flow	Observable /	Signal / computed

	BehaviorSubject	
Fetching	HttpClient.get().pipe(...)	rxResource()
Forms	FormGroup / FormControl	form() / FieldTree
Change Detection	zone.js / ChangeDetectorRef	Zoneless (Signal-driven)
Templates	`*ngIf="obs\$`	async"
Micro Frontends	Webpack Module Federation	Native Federation (ESM)

Would you like me to generate the Native Federation configuration file (federation.config.js) to help you separate your PDF and C++ engine logic into its own micro-service?

To help you visualize the transition to signals, this video explains the "Zoneless" mindset and how it simplifies complex apps like yours.

Migrating Angular to Signals and Zoneless Mode